

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_87c49086-be4\agent-a78a432b1a1f60d69.jsonl`
- SHA-256: `c5f1a5777da2c617f37fc4cba98a70bf560e638b62ccc2403ce9fb2ee64203c2`
- Source modified: `2026-06-18T08:13:06+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_87c49086-be4`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`
```

Transcript

1. user (2026-06-18T08:05:20.722Z)

You implement ONE behavior-preserving refactor and open a PR for OWNER REVIEW (do NOT merge). You are in your OWN isolated git worktree of the badminton-highlight-indexer repo on THIS machine.
The Python env has pytest/ruff/mypy/cv2/torch/numpy ? the full offline suite RUNS here (ignore any CLAUDE.md note saying it can't).

SETUP FIRST, exactly:

```
git fetch origin
git checkout -B <BRANCH> origin/master      # base MUST be origin/master, NOT the current HEAD
then verify: `git log --oneline -1` is origin/master's HEAD and `git branch --show-current` == <BRANCH>.
```

RULES (pure refactor ? quality must NOT regress, behavior must NOT change):

- Behavior-PRESERVING moves/extractions ONLY. Do NOT change logic, branch conditions, log strings/levels, return types, ordering, thresholds, or constants. It is cut-paste into helpers.
- Edit ONLY the files named in the task. Stage EXPLICIT paths (`git add <file>`), NEVER -A/./-u.
- Match the surrounding code style and naming.

GATE (binding ? run all four; capture pass/skip counts + coverage %):

```
python run_all_tests.py
python -m ruff check .
python -m mypy backend training
python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70
```

ONLY IF ALL FOUR ARE GREEN:

```
git add <explicit files>
git commit -m "<conventional commit msg>"    # final line: Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>
git push -u origin <BRANCH>
gh pr create --base master --head <BRANCH> --title "<title>" --body "<concise body: what/why, behavior-preserving argument, gate results>"    # NO merge
IF ANY GATE IS RED: do NOT push or open a PR. Fix if it's a trivial slip you introduced;
otherwise
report status='gate-failed' with the failing output and your diff so far. Never weaken a test or lower the coverage floor to go green.
```

Return the structured object with: slug, branch, status ('pr-opened'|'gate-failed'|'error'), pr_url (""+reason if none), diffstat, suite (e.g. '851 passed, 7 skipped'), ruff, mypy, coverage, behavior_note (why it's a pure move), caveat.

=== YOUR TASK: compiler-trim-extract (branch refactor/compiler-trim-one-segment) ===

FILE: backend/pipeline/stitcher/compiler.py ? method compile_highlights. Extract the body of

```
the per-segment `for idx, (raw_start, raw_end) in enumerate(...)` loop VERBATIM into a new
instance
method `_trim_one_segment(self, ...) -> (clip_path_or_None, skip_record_or_None)`.
compile_highlights
keeps the for-loop and appends to temp_clips / skipped_segments exactly as today, calling the
helper
per segment. Keep ALL clamp arithmetic, the watermark-then-plain ffmpeg retry order
(`attempts = [vf_args, []] if vf_args else [[]]`), and every skip-reason string identical.
TESTS: tests/test_compiler.py (clamp/skip/ordering/concat), tests/test_watermark.py (watermark
fallback).
```

You are running in an isolated git worktree at C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\wf_87c49086-be4-2 (a separate working copy of the repo). Changes you make here do NOT affect the main working directory (C:\Users\avidu\Projects\badminton-highlight-indexer\.claude\worktrees\refactor-base) or other agents. Work normally ? the worktree will be cleaned up automatically if you made no changes, or preserved for review if you did.

2. assistant (2026-06-18T08:05:23.095Z)

I'll start by setting up the worktree branch correctly, then examine the target file.

3. user (2026-06-18T08:05:27.652Z)

```
branch 'refactor/compiler-trim-one-segment' set up to track 'origin/master'.
Switched to a new branch 'refactor/compiler-trim-one-segment'
---HEAD---
a010667 feat(eval): nightly all-OFF floor ? pin a baseline_off reference config (#217)
---BRANCH---
refactor/compiler-trim-one-segment
```

4. user (2026-06-18T08:05:27.706Z)

```
1     import os
2     import subprocess
3     import tempfile
4     import logging
5     from pathlib import Path
6     from typing import Any, Dict, List, Optional, Tuple, Union
7
8     logger = logging.getLogger(__name__)
9
10    # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the
overlay
11    # renders out-of-the-box without a system fontconfig setup (a common gap on Windows,
where
12    # `font='Sans'` fails). Overridable via stitching.watermark.font_path. This module lives
at
13    # backend/pipeline/stitcher/, so parents[2] == backend/.
14    _BUNDLED_FONT = Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-
Regular.ttf"
15
16    class VideoCompiler:
17        def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
0.5,
18                    watermark: Optional[Any] = None):
19            self.output_dir = output_dir
20            self.end_padding = end_padding
21            self.begin_padding = begin_padding
22            # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
23            # normalized to a dict (or None) so this module stays config-package-agnostic.
24            self.watermark = self._normalize_watermark(watermark)
25            os.makedirs(output_dir, exist_ok=True)
26
```

```

27     @staticmethod
28     def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
29         if watermark is None:
30             return None
31         if hasattr(watermark, "model_dump"):
32             return watermark.model_dump()
33         if isinstance(watermark, dict):
34             return dict(watermark)
35         return None
36
37     @staticmethod
38     def _ff_quote(path: str) -> str:
39         """Make a filesystem path safe to embed inside single quotes in an ffmpeg
40         filtergraph operand (drawtext fontfile/textfile, movie= source): forward
41         slashes (Windows-friendly), escaped single quotes, AND escaped colons.
42
43         The colon MUST be escaped even inside single quotes: ffmpeg's filtergraph
44         option parser treats ':' as the option separator, so a Windows drive-letter
45         path like ``C:/dir/font.ttf`` otherwise truncates the operand at ``C`` and
46         the encode fails with "Invalid argument". This silently disabled the
47         on-by-default bundled-font watermark (and absolute logo paths) on Windows."""
48         return path.replace("\\", "/").replace("'", r"\'").replace(":", r"\:")
49
50     @staticmethod
51     def _parse_time(val: Union[int, float, str]) -> float:
52         """Normalizes a timestamp value to float seconds.
53         Handles: float/int (pass-through), plain numeric strings ("12.4"),
54         and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
55         This mirrors the parse_time logic used in gemini.py to ensure
56         the stitcher can always consume whatever format the indexer produced."""
57         if isinstance(val, (int, float)):
58             return float(val)
59         if isinstance(val, str):
60             val = val.strip()
61             if ":" in val:
62                 # Handle MM:SS, HH:MM:SS, etc.
63                 parts = val.split(":")
64                 parts.reverse()
65                 total = 0.0
66                 for i, p in enumerate(parts):
67                     try:
68                         total += float(p) * (60 ** i)
69                     except ValueError:
70                         pass
71                 return total
72             try:
73                 return float(val)
74             except ValueError:
75                 logger.warning(f"Could not parse timestamp value '{val}' as a number.
76                 Defaulting to 0.0.")
76                 return 0.0
77                 logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
78                 '{val}'. Defaulting to 0.0.")
78                 return 0.0
79
80     @staticmethod
81     def _merge_overlapping(segments: List[Tuple[float, float]]) -> List[Tuple[float,
82     float]]:
83         """Normalize, sort, and merge overlapping/duplicate (start, end) ranges.
84
85         A highlight reel must never replay the same footage. The same rally can land in
86         the segment list more than once ? e.g. two detector runs accumulated for one
87         video (add_video is an UPSERT, so re-running without a prune leaves both sets),
88         or
89         chunk-overlap duplicates near boundaries. Stitching those verbatim plays the

```

```

rally
88         twice. Merging any range that overlaps or abuts the previous one collapses true
89         duplicates/overlaps while preserving genuinely distinct (gapped) rallies.
90         Degenerate rows (end <= start) are dropped. Timestamps may be float/int/str.
91         """
92         parsed = []
93         for raw_s, raw_e in segments:
94             s = VideoCompiler._parse_time(raw_s)
95             e = VideoCompiler._parse_time(raw_e)
96             if e > s:
97                 parsed.append((s, e))
98         parsed.sort()
99         merged: List[Tuple[float, float]] = []
100         for s, e in parsed:
101             if merged and s <= merged[-1][1]: # overlaps/abuts previous -> extend it
102                 merged[-1] = (merged[-1][0], max(merged[-1][1], e))
103             else:
104                 merged.append((s, e))
105         return merged
106
107     def _get_video_duration(self, video_path: str) -> float:
108         """Probes the video file to get its total duration in seconds.
109         Returns 0.0 if it cannot be determined."""
110         try:
111             result = subprocess.run(
112                 [
113                     "ffprobe", "-v", "error",
114                     "-show_entries", "format=duration",
115                     "-of", "default=noprint_wrappers=1:nokey=1",
116                     video_path
117                 ],
118                 capture_output=True, text=True, check=True
119             )
120             return float(result.stdout.strip())
121         except Exception as e:
122             logger.warning(f"Could not determine video duration via ffprobe: {e}")
123             return 0.0
124
125     def _watermark_filter(self, tmp_dir: str) -> Optional[str]:
126         """Build the ffmpeg -vf filtergraph that burns the configured watermark (text
127         and/or logo PNG) into a clip during the per-clip re-encode. Returns None when
128         watermarking is disabled or has nothing to draw. The concat stage stays -c copy
129         because the mark is already baked into every clip identically."""
130         wm = self.watermark
131         if not wm or not wm.get("enabled"):
132             return None
133
134         text = (wm.get("text") or "").strip()
135         logo_path = (wm.get("logo_path") or "").strip()
136         if logo_path and not os.path.exists(logo_path):
137             logger.warning(f"[watermark] logo not found: {logo_path} ? skipping logo
overlay.")
138             logo_path = ""
139         if not text and not logo_path:
140             return None
141
142         margin = int(wm.get("margin", 24))
143         opacity = float(wm.get("opacity", 0.85))
144         position = str(wm.get("position", "bottom-right"))
145         # drawtext uses lowercase w/h (main) + tw/th (text); overlay uses W/H (main) +
w/h (logo).
146         dt_xy = {
147             "bottom-right": f"x=w-tw-{margin}:y=h-th-{margin}",
148             "bottom-left": f"x={margin}:y=h-th-{margin}",
149             "top-right": f"x=w-tw-{margin}:y={margin}",

```

```

150         "top-left": f"x={margin}:y={margin}",
151     }.get(position, f"x=w-tw-{margin}:y=h-th-{margin}")
152     ov_xy = {
153         "bottom-right": f"W-w-{margin}:H-h-{margin}",
154         "bottom-left": f"{margin}:H-h-{margin}",
155         "top-right": f"W-w-{margin}:{margin}",
156         "top-left": f"{margin}:{margin}",
157     }.get(position, f"W-w-{margin}:H-h-{margin}")
158
159     drawtext = ""
160     if text:
161         ratio = float(wm.get("font_size_ratio") or 0.035)
162         divisor = max(1, round(1.0 / ratio))
163         # textfile= sidesteps the notorious drawtext text= escaping for arbitrary
strings.
164         tf_path = os.path.join(tmp_dir, "watermark.txt")
165         with open(tf_path, "w", encoding="utf-8") as fh:
166             fh.write(text)
167         # Font precedence: explicit font_path ? bundled Apache-2.0 default (renders
168         # without fontconfig) ? fontconfig family (last-resort, environment-
dependent).
169         font_path = (wm.get("font_path") or "").strip()
170         if not font_path and _BUNDLED_FONT.exists():
171             font_path = str(_BUNDLED_FONT)
172         if font_path:
173             font_opt = f"fontfile='{self._ff_quote(font_path)}'"
174         else:
175             font_opt = f"font='{wm.get('font', 'Sans')}'"
176         box = ""
177         if wm.get("box", True):
178             box = f"box=1:boxcolor=black@{min(opacity, 0.5):.2f}:boxborderw=10"
179         drawtext = (
180             f"drawtext=textfile='{self._ff_quote(tf_path)}':{font_opt}"
181             f"fontcolor=white@{opacity:.2f}:fontsize=h/{divisor}:{dt_xy}{box}"
182         )
183
184         if logo_path and drawtext:
185             return
186         f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy},{drawtext}"
187         if logo_path:
188             return f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy}"
189         return drawtext
190
191     def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float,
float]], output_filename: str) -> str:
192         """
193         Extracts selected segments from a raw video and stitches them together
194         into a seamless highlights file, applying end_padding to prevent abrupt endings.
195         """
196         if not segments:
197             raise ValueError("No highlight segments provided to compile.")
198
199         # Never stitch the same footage twice: collapse overlapping/duplicate ranges
200         # (e.g. accumulated re-run segments or chunk-overlap dupes) before trimming.
201         original_count = len(segments)
202         segments = self._merge_overlapping(segments)
203         if len(segments) < original_count:
204             logger.info("Merged %d overlapping/duplicate segment(s) before stitching: %d
-> %d.",
205                 original_count - len(segments), original_count, len(segments))
206         if not segments:
207             raise ValueError("No valid (start < end) highlight segments after merge.")
208
209         if not os.path.exists(raw_video_path):
210             raise FileNotFoundError(f"[COMPILER ERROR] Source video file not found:

```

```

{raw_video_path}")
210
211     # Defense-in-depth: never let the output name escape output_dir (the API also
212     # validates this at the request boundary).
213     output_path = os.path.join(self.output_dir, os.path.basename(output_filename))
214
215     # Probe video duration so we can detect out-of-range segments
216     video_duration = self._get_video_duration(raw_video_path)
217     if video_duration > 0:
218         logger.info(f"Source video duration: {video_duration:.2f}s")
219     else:
220         logger.warning("Video duration unknown. Cannot validate segment boundaries
against video length.")
221
222     # We will create temporary clips and stitch them using FFmpeg concat demuxer
223     temp_clips = []
224     skipped_segments = []
225
226     # Create a temp directory within output_dir
227     with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
228         logger.info(f"Trimming {len(segments)} segments
(begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
229         # Build the (optional) watermark filtergraph once; it is applied during
every
230         # per-clip re-encode so the mark is baked into each clip identically.
231         watermark_filter = self._watermark_filter(tmp_dir)
232         if watermark_filter:
233             logger.info("[watermark] applying branding overlay to each clip.")
234         for idx, (raw_start, raw_end) in enumerate(segments):
235             # Normalize timestamps to float seconds ? handles float, int,
236             # plain numeric strings, and colon-separated formats like "MM:SS"
237             start = self._parse_time(raw_start)
238             end = self._parse_time(raw_end)
239             segment_label = f"Segment #{idx+1}/{len(segments)} [{start:.2f}s -
{end:.2f}s]"
240
241             # Validate the segment times
242             if start < 0:
243                 logger.warning(f"{segment_label}: start_time is negative
({start:.2f}s). Clamping to 0.")
244                 start = 0.0
245
246             if start >= end:
247                 logger.error(f"{segment_label}: start_time ({start:.2f}s) >=
end_time ({end:.2f}s). Skipping invalid segment.")
248                 skipped_segments.append((idx, start, end, "start >= end"))
249                 continue
250
251             # Check if segment extends beyond video duration
252             if video_duration > 0:
253                 if start >= video_duration:
254                     logger.error(f"{segment_label}: start_time ({start:.2f}s) is
beyond the video duration ({video_duration:.2f}s). "
255                                 f"This segment cannot be extracted ? the video ran
out. Skipping.")
256                     skipped_segments.append((idx, start, end, "start beyond video
end"))
257                     continue
258
259                 if end > video_duration:
260                     original_end = end
261                     end = video_duration
262                     logger.warning(f"{segment_label}: end_time ({original_end:.2f}s)
exceeds video duration ({video_duration:.2f}s). "
263                                 f"Clamping end_time to {video_duration:.2f}s. The

```

```

segment may be truncated.")
264
265         clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")
266
267         # Precise sub-second trim using fast re-encoding to preserve stream
headers
268         padded_start = max(0.0, start - self.begin_padding)
269         padded_end = end + self.end_padding
270
271         # Clamp padded_end to video duration if known
272         if video_duration > 0 and padded_end > video_duration:
273             padded_end = video_duration
274             logger.info(f"{segment_label}: Padded end ({end +
self.end_padding:.2f}s) exceeds video duration. "
275                       f"Clamping to {video_duration:.2f}s (end padding
partially applied).")
276
277         trim_duration = padded_end - padded_start
278         logger.info(f"Trimming {segment_label} -> [{padded_start:.3f}s -
{padded_end:.3f}s] (duration: {trim_duration:.2f}s)")
279
280         base_cmd = [
281             "ffmpeg", "-y",
282             "-ss", str(round(padded_start, 3)),
283             "-to", str(round(padded_end, 3)),
284             "-i", raw_video_path,
285             "-c:v", "libx264",
286             "-preset", "superfast",
287             "-crf", "22",
288             "-c:a", "aac",
289             "-b:a", "128k",
290         ]
291         vf_args = ["-vf", watermark_filter] if watermark_filter else []
292
293         # Try with the watermark first; if that encode fails (e.g. a bad
font/logo
294         # path), retry once WITHOUT it so a branding misconfig can never nuke
the
295         # whole reel. Clips stay codec-identical either way, so concat -c copy
holds.
296         attempts = [vf_args, []] if vf_args else [[]]
297         produced = False
298         last_exc: Optional[subprocess.CallledProcessError] = None
299         cmd = base_cmd + [clip_path]
300         for attempt_vf in attempts:
301             cmd = base_cmd + attempt_vf + [clip_path]
302             try:
303                 subprocess.run(cmd, capture_output=True, check=True)
304             except subprocess.CallledProcessError as e:
305                 last_exc = e
306                 if attempt_vf and vf_args:
307                     _err = (e.stderr.decode(errors="replace").strip()[-300:]
308                             if getattr(e, "stderr", None) else "")
309                     logger.warning(
310                         f"{segment_label}: watermark encode failed; retrying
without it. "
311                         f"ffmpeg: {_err or '(no stderr captured)'}"
312                     )
313                     continue
314                 break
315             if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
316                 produced = True
317                 if not attempt_vf and vf_args:
318                     logger.warning(f"{segment_label}: encoded WITHOUT the
watermark (the watermark filter failed).")

```

```

319             break
320             last_exc = None # ffmpeg succeeded but produced nothing
321             break
322
323         if produced:
324             temp_clips.append(clip_path)
325         elif last_exc is not None:
326             stderr_msg = last_exc.stderr.decode(errors='replace').strip()
327             logger.error(f"{segment_label}: FFmpeg trim failed.")
328             print(f"  Command: {' '.join(cmd)}")
329             print(f"  FFmpeg stderr: {stderr_msg[-500:]}") # Last 500 chars to
avoid flooding
330             skipped_segments.append((idx, start, end, f"ffmpeg error:
{stderr_msg[-200:]}"))
331             continue
332         else:
333             logger.error(f"{segment_label}: FFmpeg exited successfully but
output clip is missing or empty. Skipping.")
334             skipped_segments.append((idx, start, end, "empty output clip"))
335             continue
336
337         # Report summary of skipped segments
338         if skipped_segments:
339             logger.info(f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:")
340             for seg_idx, seg_start, seg_end, reason in skipped_segments:
341                 logger.info(f"  - Segment #{seg_idx+1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}")
342                 logger.info("")
343
344         if not temp_clips:
345             raise RuntimeError(
346                 f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
347                 f"All {len(segments)} segments were skipped. "
348                 f"Check the errors above for details. "
349                 f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
350             )
351
352         logger.info(f"Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
Preparing to stitch...")
353
354         # Create a list file for FFmpeg concat demuxer
355         list_file_path = os.path.join(tmp_dir, "clips.txt")
356         with open(list_file_path, "w") as f:
357             for clip in temp_clips:
358                 # Escape single quotes in path
359                 safe_path = clip.replace("'", "'\\'")
360                 f.write(f"file '{safe_path}'\n")
361
362         logger.info("Stitching clips together...")
363         # Stitch them together without re-encoding (since they were encoded with
identical settings)
364         concat_cmd = [
365             "ffmpeg", "-y",
366             "-f", "concat",
367             "-safe", "0",
368             "-i", list_file_path,
369             "-c", "copy",
370             output_path
371         ]
372
373         try:
374             subprocess.run(concat_cmd, capture_output=True, check=True)
375

```

```

376         # Final verification
377         if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
378             raise RuntimeError(
379                 f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
380                 f"but output file is missing or empty: {output_path}"
381             )
382
383         output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
384         logger.info(f"Dynamic highlight reel saved: {output_path}
385         ({output_size_mb:.1f} MB)")
386         return output_path
387     except subprocess.CalledProcessError as e:
388         stderr_msg = e.stderr.decode(errors='replace').strip()
389         raise RuntimeError(
390             f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
391             f"  Command: {' '.join(concat_cmd)}\n"
392             f"  FFmpeg stderr: {stderr_msg[-500:]}")
393     ) from e

```

5. assistant (2026-06-18T08:05:30.713Z)

The branch is set up correctly (HEAD is origin/master's a010667, branch is refactor/compiler-trim-one-segment). Now let me examine the relevant tests to understand what behavior must be preserved.

6. user (2026-06-18T08:05:31.975Z)

```

1      """VideoCompiler coverage (previously zero ? the user-facing highlight-reel output path,
2      where the f-string SyntaxError lived undetected). ffmpeg/ffprobe are mocked."""
3      import os
4      import subprocess
5      from unittest.mock import patch, MagicMock
6
7      import pytest
8
9      from backend.pipeline.stitcher.compiler import VideoCompiler
10
11
12      def test_parse_time_formats():
13          f = VideoCompiler._parse_time
14          assert f(12.5) == 12.5
15          assert f("12.5") == 12.5
16          assert f("1:30") == 90.0
17          assert f("1:00:00") == 3600.0
18          assert f("garbage") == 0.0
19
20
21      def test_merge_overlapping_collapses_duplicates_and_overlaps():
22          m = VideoCompiler._merge_overlapping
23          # Exact duplicates + near-duplicates (the "every rally twice" re-run bug) collapse
24          to one each.
25          assert m([(28.2, 34.5), (28.5, 34.5), (40.5, 45.5), (40.5, 45.5)]) == [(28.2, 34.5),
26          (40.5, 45.5)]
27          # Unsorted input is sorted; an overlapping range extends the previous.
28          assert m([(100.0, 110.0), (0.0, 5.0), (104.0, 120.0)]) == [(0.0, 5.0), (100.0,
29          120.0)]
30          # Genuinely distinct (gapped) rallies are preserved, not merged.
31          assert m([(0.0, 5.0), (6.0, 9.0)]) == [(0.0, 5.0), (6.0, 9.0)]
32          # Degenerate rows (end <= start) are dropped; string timestamps are accepted.
33          assert m(["0:10", "0:20", (30.0, 30.0), (31.0, 29.0)]) == [(10.0, 20.0)]
34
35      def test_compile_dedups_before_stitching(tmp_path):
36          """A segment list with every rally twice must yield ONE clip per unique rally."""

```

```

35     comp = VideoCompiler(str(tmp_path / "out"))
36     raw = tmp_path / "raw.mp4"
37     raw.write_bytes(b"x")
38     trims = []
39
40     def fake_run(cmd, *a, **k):
41         if cmd[0] == "ffprobe":
42             return MagicMock(stdout="100.0\n", returncode=0)
43         out = cmd[-1]
44         if isinstance(out, str) and out.endswith(".mp4"):
45             open(out, "wb").write(b"clip")
46             # count per-clip trims (the concat list file input is not an .mp4 output)
47             if "-i" in cmd and cmd.index("-i") + 1 == str(raw):
48                 trims.append(cmd)
49             return MagicMock(returncode=0, stdout=b"", stderr=b"")
50
51     dup_segments = [(0.0, 5.0), (0.2, 5.0), (10.0, 15.0), (10.0, 15.0), (20.0, 25.0)]
52     with patch.object(subprocess, "run", side_effect=fake_run):
53         comp.compile_highlights(str(raw), dup_segments, "reel.mp4")
54     # 5 input segments collapse to 3 unique rallies -> 3 per-clip trims, not 5.
55     assert len(trims) == 3, f"expected 3 deduped trims, got {len(trims)}"
56
57
58     def test_compile_basenames_output_and_stitches(tmp_path):
59         comp = VideoCompiler(str(tmp_path / "out"))
60         raw = tmp_path / "raw.mp4"
61         raw.write_bytes(b"x")
62
63         calls = []
64
65         def fake_run(cmd, *a, **k):
66             calls.append(cmd)
67             # Create whatever output file ffmpeg/ffprobe was told to write so existence
68             if cmd[0] == "ffprobe":
69                 return MagicMock(stdout="100.0\n", returncode=0)
70             out = cmd[-1]
71             if isinstance(out, str) and out.endswith(("mp4",)):
72                 open(out, "wb").write(b"clip")
73             return MagicMock(returncode=0, stdout=b"", stderr=b"")
74
75         with patch.object(subprocess, "run", side_effect=fake_run):
76             # Traversal in the name must be stripped (defense-in-depth basename).
77             out_path = comp.compile_highlights(str(raw), [(0.0, 5.0), (10.0, 15.0)],
78             "../reel.mp4")
79
80             assert os.path.basename(out_path) == "reel.mp4"
81             assert os.path.dirname(out_path).endswith("out")
82             # Per-clip trims + one concat = at least 3 ffmpeg invocations (plus an ffprobe).
83             assert sum(1 for c in calls if c[0] == "ffmpeg") >= 3
84
85     def test_compile_raises_on_no_segments(tmp_path):
86         comp = VideoCompiler(str(tmp_path / "out"))
87         raw = tmp_path / "raw.mp4"
88         raw.write_bytes(b"x")
89         with pytest.raises(ValueError):
90             comp.compile_highlights(str(raw), [], "reel.mp4")
91
92
93     def test_compile_raises_when_source_missing(tmp_path):
94         comp = VideoCompiler(str(tmp_path / "out"))
95         with pytest.raises(FileNotFoundError):
96             comp.compile_highlights(str(tmp_path / "nope.mp4"), [(0.0, 5.0)], "reel.mp4")
97

```

7. user (2026-06-18T08:05:32.477Z)

```
1      """Tests for the configurable highlight watermark (stitching.watermark).
2
3      Covers the config defaults, the filtergraph builder (text + logo, positions, font,
4      the configurable string), and the end-to-end wiring through compile_highlights with
5      ffmpeg/ffprobe mocked ? including the graceful fallback when the watermark encode fails.
6      """
7      import os
8      import subprocess
9      from pathlib import Path
10     from unittest.mock import MagicMock, patch
11
12     import pytest
13     from pydantic import ValidationError
14
15     from backend.config import AppConfig
16     from backend.config.models import WatermarkConfig
17     from backend.pipeline.stitcher.compiler import VideoCompiler
18
19
20     # --- config -----
21
22     def test_watermark_config_defaults_on_with_brand_text():
23         wm = AppConfig().stitching.watermark
24         assert wm.enabled is True # default-on, with an off switch
25         assert wm.text == "Generated by Khelsutra.guru"
26         assert wm.position == "bottom-right"
27
28
29     def test_watermark_can_be_disabled_via_config():
30         cfg = AppConfig.model_validate({"stitching": {"watermark": {"enabled": False}}})
31         assert cfg.stitching.watermark.enabled is False
32
33
34     def test_font_size_ratio_and_opacity_bounds_enforced():
35         with pytest.raises(ValidationError):
36             WatermarkConfig(opacity=1.5)
37         with pytest.raises(ValidationError):
38             WatermarkConfig(font_size_ratio=0.0)
39
40
41     # --- filtergraph builder -----
42
43     def test_disabled_or_none_builds_no_filter(tmp_path):
44         assert VideoCompiler(str(tmp_path / "a"),
45 watermark=None)._watermark_filter(str(tmp_path)) is None
46         off = VideoCompiler(str(tmp_path / "b"), watermark={"enabled": False})
47         assert off._watermark_filter(str(tmp_path)) is None
48
49     def test_text_filter_writes_configurable_string_and_builds_drawtext(tmp_path):
50         custom = "My Academy 2026 ? Khelsutra.guru"
51         comp = VideoCompiler(str(tmp_path / "o"), watermark={"enabled": True, "text":
52 custom})
53         graph = comp._watermark_filter(str(tmp_path))
54         assert graph is not None and graph.startswith("drawtext=")
55         # The arbitrary, configurable string is written verbatim to the textfile (no
56 escaping pain).
57         assert (tmp_path / "watermark.txt").read_text(encoding="utf-8") == custom
58         assert "textfile=" in graph
59         assert "fontcolor=white@0.85" in graph
60         assert "fontsize=h/29" in graph # round(1/0.035) == 29
61         assert "x=w-tw-24:y=h-th-24" in graph # bottom-right default, margin 24
62         assert "box=1" in graph
```

```

61
62
63     def test_position_and_margin_map_to_corner(tmp_path):
64         comp = VideoCompiler(str(tmp_path / "o"),
65                               watermark={"enabled": True, "text": "X", "position": "top-
left", "margin": 12})
66         graph = comp._watermark_filter(str(tmp_path))
67         assert "x=12:y=12" in graph
68
69
70     def test_explicit_font_path_uses_fontfile(tmp_path):
71         comp = VideoCompiler(str(tmp_path / "o1"),
72                               watermark={"enabled": True, "text": "X", "font_path":
"/fonts/Brand.ttf"})
73         g = comp._watermark_filter(str(tmp_path))
74         assert "fontfile='/fonts/Brand.ttf'" in g
75
76
77     def test_default_uses_bundled_apache_font(tmp_path):
78         # No font_path ? the bundled Apache-2.0 Roboto Slab (renders without fontconfig).
79         comp = VideoCompiler(str(tmp_path / "o2"), watermark={"enabled": True, "text": "X"})
80         g = comp._watermark_filter(str(tmp_path))
81         assert "fontfile=" in g and "RobotoSlab-Regular.ttf" in g
82         assert "font=" not in g # not the fontconfig path
83
84
85     def test_bundled_font_actually_ships_and_is_apache():
86         import backend.pipeline.stitcher.compiler as cmod
87         assert cmod._BUNDLED_FONT.exists(), "bundled default watermark font must ship in the
repo"
88         with open(cmod._BUNDLED_FONT, "rb") as fh:
89             assert fh.read(4) in (b"\x00\x01\x00\x00", b"OTTO", b"true", b"ttcf") # valid
sfnt
90             lic = cmod._BUNDLED_FONT.parent / "LICENSE-Apache-2.0.txt"
91             assert lic.exists() and "Apache License" in lic.read_text(encoding="utf-8",
errors="replace")
92
93
94     def test_fontconfig_fallback_when_bundled_missing(tmp_path, monkeypatch):
95         import backend.pipeline.stitcher.compiler as cmod
96         monkeypatch.setattr(cmod, "_BUNDLED_FONT", Path("/no/such/font.ttf"))
97         comp = VideoCompiler(str(tmp_path / "o3"),
98                               watermark={"enabled": True, "text": "X", "font": "DejaVu
Sans"})
99         g = comp._watermark_filter(str(tmp_path))
100         assert "font='DejaVu Sans'" in g and "fontfile=" not in g
101
102
103     # --- Windows drive-letter colon escaping (regression) -----
104     # A Windows absolute path's drive-letter colon (C:/...) must be escaped for the
105     # ffmpeg filtergraph, or drawtext/movie parse ':' as an option separator and the
106     # encode fails with "Invalid argument" ? which silently disabled the on-by-default
107     # bundled-font watermark (and absolute logo paths) on Windows. Verified end-to-end
108     # against real ffmpeg before this fix landed.
109
110     def test_ff_quote_escapes_drive_letter_colon():
111         assert VideoCompiler._ff_quote(r"C:\fonts\Brand.ttf") == r"C\:/fonts/Brand.ttf"
112         # no colon -> unchanged (POSIX paths and the Linux bundled font are unaffected)
113         assert VideoCompiler._ff_quote("/usr/share/fonts/font.ttf") ==
"/usr/share/fonts/font.ttf"
114         # colon + single-quote + backslash all handled
115         assert VideoCompiler._ff_quote(r"D:\a\b\f.ttf") == r"D\:/a\b/f.ttf"
116
117
118     def test_watermark_font_path_with_drive_colon_is_escaped(tmp_path):

```

```

119     # Explicit font_path (literal, existence not required) -> deterministic on any OS.
120     comp = VideoCompiler(str(tmp_path / "o"),
121                           watermark={"enabled": True, "text": "X", "font_path":
"C:/fonts/Brand.ttf"})
122     g = comp._watermark_filter(str(tmp_path))
123     assert r"fontfile='C:/fonts/Brand.ttf'" in g
124     assert "fontfile='C:/fonts" not in g # the raw colon would break drawtext parsing
125
126
127     def test_logo_plus_text_overlays_then_draws(tmp_path):
128         logo = tmp_path / "logo.png"
129         logo.write_bytes(b"\x89PNG\r\n")
130         comp = VideoCompiler(str(tmp_path / "o"),
131                               watermark={"enabled": True, "text": "X", "logo_path":
str(logo)})
132         graph = comp._watermark_filter(str(tmp_path))
133         assert graph.startswith("movie=")
134         assert "overlay=W-w-24:H-h-24" in graph
135         assert "drawtext=" in graph
136
137
138     def test_missing_logo_is_skipped_gracefully(tmp_path):
139         comp = VideoCompiler(str(tmp_path / "o"),
140                               watermark={"enabled": True, "text": "X", "logo_path":
str(tmp_path / "nope.png")})
141         graph = comp._watermark_filter(str(tmp_path))
142         assert "movie=" not in graph and graph.startswith("drawtext=") # text still drawn
143
144
145     # --- end-to-end wiring (ffmpeg/ffprobe mocked) -----
146
147     def _fake_run_factory(calls, fail_on_vf=False):
148         def fake_run(cmd, *a, **k):
149             calls.append(cmd)
150             if cmd[0] == "ffprobe":
151                 return MagicMock(stdout="100.0\n", returncode=0)
152             if fail_on_vf and "-vf" in cmd:
153                 raise subprocess.CalledProcessError(1, cmd, output=b"", stderr=b"font not
found")
154             out = cmd[-1]
155             if isinstance(out, str) and out.endswith(".mp4"):
156                 open(out, "wb").write(b"clip")
157             return MagicMock(returncode=0, stdout=b"", stderr=b"")
158         return fake_run
159
160
161     def test_compile_applies_watermark_vf_to_each_clip_not_concat(tmp_path):
162         raw = tmp_path / "raw.mp4"
163         raw.write_bytes(b"x")
164         comp = VideoCompiler(str(tmp_path / "out"),
165                               watermark={"enabled": True, "text": "Generated by
Khelsutra.guru"})
166         calls = []
167         with patch.object(subprocess, "run", side_effect=_fake_run_factory(calls)):
168             comp.compile_highlights(str(raw), [(0.0, 5.0), (10.0, 15.0)], "reel.mp4")
169
170         trim_cmds = [c for c in calls if c[0] == "ffmpeg" and "-i" in c and "concat" not in
c]
171         assert trim_cmds, "expected per-clip trim invocations"
172         for c in trim_cmds:
173             assert "-vf" in c
174             assert "drawtext=" in c[c.index("-vf") + 1]
175         # The concat (-c copy) must NOT re-filter ? the mark is already baked into each
clip.
176         concat_cmds = [c for c in calls if "concat" in c]

```

```

177         assert concat_cmds and all("-vf" not in c for c in concat_cmds)
178
179
180     def test_compile_without_watermark_adds_no_vf(tmp_path):
181         raw = tmp_path / "raw.mp4"
182         raw.write_bytes(b"x")
183         comp = VideoCompiler(str(tmp_path / "out")) # no watermark passed
184         calls = []
185         with patch.object(subprocess, "run", side_effect=_fake_run_factory(calls)):
186             comp.compile_highlights(str(raw), [(0.0, 5.0)], "reel.mp4")
187         assert all("-vf" not in c for c in calls)
188
189
190     def test_watermark_failure_falls_back_to_unwatermarked_clip(tmp_path):
191         """A bad font/logo must not nuke the reel: the clip re-encodes without the
192 watermark."""
193         raw = tmp_path / "raw.mp4"
194         raw.write_bytes(b"x")
195         comp = VideoCompiler(str(tmp_path / "out"),
196                               watermark={"enabled": True, "text": "Generated by
197 Khelsutra.guru"})
198         calls = []
199         with patch.object(subprocess, "run", side_effect=_fake_run_factory(calls,
200 fail_on_vf=True)):
201             out_path = comp.compile_highlights(str(raw), [(0.0, 5.0)], "reel.mp4")
202
203         # Reel still produced despite the watermark encode failing.
204         assert out_path.endswith("reel.mp4")
205         assert os.path.exists(out_path)
206         # For the clip, both a watermarked attempt and a plain fallback attempt were made.
207         clip_cmds = [c for c in calls if c[0] == "ffmpeg" and "concat" not in c and "-i" in
208 c]
209         assert any("-vf" in c for c in clip_cmds) # watermark attempt
210         assert any("-vf" not in c for c in clip_cmds) # fallback attempt
211
212
213     def test_watermark_failure_logs_the_ffmpeg_reason(tmp_path, caplog):
214         """The fallback warning must surface ffmpeg's stderr (the *why*), not go silent ?
215 e.g. the 'Fontconfig error' that tells the user to set font_path."""
216         import logging as _logging
217         raw = tmp_path / "raw.mp4"
218         raw.write_bytes(b"x")
219         comp = VideoCompiler(str(tmp_path / "out"), watermark={"enabled": True, "text":
220 "y"})
221
222         def fake_run(cmd, *a, **k):
223             if cmd[0] == "ffprobe":
224                 return MagicMock(stdout="100.0\n", returncode=0)
225             if "-vf" in cmd:
226                 raise subprocess.CalledProcessError(
227                     1, cmd, output=b"", stderr=b"Fontconfig error: Cannot load default
228 config file")
229             out = cmd[-1]
230             if isinstance(out, str) and out.endswith(".mp4"):
231                 open(out, "wb").write(b"clip")
232             return MagicMock(returncode=0, stdout=b"", stderr=b"")
233
234         with caplog.at_level(_logging.WARNING):
235             with patch.object(subprocess, "run", side_effect=fake_run):
236                 comp.compile_highlights(str(raw), [(0.0, 5.0)], "reel.mp4")
237         assert "Fontconfig error" in caplog.text
238

```

Now I have full understanding. I'll extract the loop body verbatim into `\_trim\_one\_segment`. The helper needs these inputs that the loop body reads: `idx`, `raw\_start`, `raw\_end`, `segments` (for `len(segments)` in the label), `video\_duration`, `tmp\_dir`, `raw\_video\_path`, `watermark\_filter`. The loop body's two outcomes are: append `clip\_path` to `temp\_clips`, or append a tuple to `skipped\_segments`. `continue` statements become `return`.

Let me design the helper to return `(clip\_path\_or\_None, skip\_record\_or\_None)`. The `continue` paths return `(None, skip\_record)`; the success path returns `(clip\_path, None)`.

9. user (2026-06-18T08:06:05.777Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\claude\worktrees\wf_87c49086-be4-2\backend\pipeline\stitcher\compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

10. assistant (2026-06-18T08:06:10.679Z)

Now I'll add the `\_trim\_one\_segment` helper method. I'll place it just before `compile\_highlights`. The body is the loop body verbatim, with `continue` -> `return (None, skip\_record)` and the success-append -> `return (clip\_path, None)`. Each segment goes through exactly one outcome (the original loop body had each path either `continue` or fall through to append exactly one clip), so returning a `(clip, skip)` pair preserves the exact semantics: in the original, a segment either appends a clip OR appends a skip record, never both ? so one of the two returned values is always None.

11. user (2026-06-18T08:06:29.300Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\claude\worktrees\wf_87c49086-be4-2\backend\pipeline\stitcher\compiler.py has been updated successfully. (file state is current in your context ? no need to Read it back)

12. assistant (2026-06-18T08:06:31.709Z)

Let me verify the edited region reads correctly around the loop.

13. user (2026-06-18T08:06:32.773Z)

```
344         watermark_filter = self._watermark_filter(tmp_dir)
345         if watermark_filter:
346             logger.info("[watermark] applying branding overlay to each clip.")
347         for idx, (raw_start, raw_end) in enumerate(segments):
348             clip_path, skip_record = self._trim_one_segment(
349                 idx, raw_start, raw_end, segments, video_duration,
350                 tmp_dir, raw_video_path, watermark_filter,
351             )
352             if clip_path is not None:
353                 temp_clips.append(clip_path)
354             if skip_record is not None:
355                 skipped_segments.append(skip_record)
356
357         # Report summary of skipped segments
358         if skipped_segments:
359             logger.info(f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:")
360             for seg_idx, seg_start, seg_end, reason in skipped_segments:
361                 logger.info(f"  - Segment #{seg_idx+1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}")
362             logger.info("")
363
364         if not temp_clips:
365             raise RuntimeError(
```

14. assistant (2026-06-18T08:06:38.760Z)

The refactor looks correct. One subtle behavior point: in the original loop, each segment that produced a clip appended exactly one clip and never a skip record (the `if produced: temp\_clips.append` branch had no `continue`, but it was the end of the loop body so it naturally moved to the next iteration). All skip paths used `continue`. So a segment is exactly one of: clip-produced (clip_path set, no skip) or skipped (skip_record set, no clip). My pair-return with two independent `if` checks preserves this exactly since the helper always returns with one slot None.

Now let me run the gate. First the test suite, ruff, and mypy.

15. user (2026-06-18T08:08:44.768Z)

```
=====
Full test suite: C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m pytest
-q
obsreport (optional reporting dep): available
=====
.....SS..... [ 8%]
..... [ 16%]
..... [ 25%]
..... [ 33%]
..... [ 41%]
..... [ 50%]
..... [ 58%]
..... [ 66%]
..... [ 75%]
.....SSSSS..... [ 83%]
..... [ 91%]
..... [100%]
856 passed, 7 skipped in 119.41s (0:01:59)
=====
RESULT: PASS - all tests green
=====
```

16. assistant (2026-06-18T08:08:49.768Z)

Suite is green (856 passed, 7 skipped). Now ruff and mypy.

17. user (2026-06-18T08:10:06.291Z)

```
=== RUFF ===
All checks passed!
=== MYPY ===
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 104 source files
```

18. assistant (2026-06-18T08:10:09.242Z)

Ruff clean, mypy clean (only informational notes, "Success: no issues found"). Now the coverage gate.

19. user (2026-06-18T08:12:12.185Z)

backend\reporting\run_signals.py	14	0	100%
backend\results.py	27	0	100%
backend\sports__init__.py	17	1	94%
backend\sports\badminton.py	13	1	92%

backend\sports\base.py	12	3	75%
backend\tools__init__.py	0	0	100%
backend\tools\export.py	75	2	97%
backend\tools\rally_slicer.py	104	47	55%
backend\utils__init__.py	0	0	100%
backend\utils\geometry.py	29	0	100%
backend\utils\hashing.py	9	0	100%
backend\utils\logging_setup.py	11	0	100%
backend\utils\paths.py	27	3	89%
backend\utils\run_telemetry.py	126	12	90%
backend\utils\video.py	25	3	88%
training__init__.py	0	0	100%
training\gen0__init__.py	0	0	100%
training\gen0\harness.py	108	11	90%

TOTAL	8011	1541	81%

Required test coverage of 70% reached. Total coverage: 80.76%

856 passed, 7 skipped, 7 warnings in 116.52s (0:01:56)

=====

RESULT: PASS - all tests green

=====

20. assistant (2026-06-18T08:12:15.395Z)

All four gates green: 856 passed / 7 skipped, ruff clean, mypy clean, coverage 80.76% (well above the 70% floor). Let me check the compiler.py coverage specifically and confirm the diff.

21. user (2026-06-18T08:12:16.589Z)

```
backend/pipeline/stitcher/compiler.py | 220 ++++++-----
1 file changed, 120 insertions(+), 100 deletions(-)
---
M backend/pipeline/stitcher/compiler.py
```

22. assistant (2026-06-18T08:12:18.842Z)

Only the target file is modified. Let me re-verify HEAD/branch right before committing (per the shared-checkout safety protocol), then commit, push, and open the PR.

23. user (2026-06-18T08:12:20.803Z)

```
BRANCH: refactor/compiler-trim-one-segment
HEAD: a010667 feat(eval): nightly all-OFF floor ? pin a baseline_off reference config (#217)
--- commits ahead of origin/master ---
```

24. assistant (2026-06-18T08:12:23.185Z)

Branch is correct, HEAD is origin/master's tip, no sibling commits ahead. Now commit the single explicit file, push, and open the PR.

25. user (2026-06-18T08:12:28.803Z)

```
[refactor/compiler-trim-one-segment 2f30631] refactor(stitcher): extract per-segment trim into
_trim_one_segment
1 file changed, 120 insertions(+), 100 deletions(-)
```

26. user (2026-06-18T08:12:34.547Z)

```
remote:
remote: Create a pull request for 'refactor/compiler-trim-one-segment' on GitHub by visiting:
remote:   https://github.com/avidullu/badminton-highlight-indexer/pull/new/refactor/compiler-
remote: trim-one-segment
remote:
branch 'refactor/compiler-trim-one-segment' set up to track 'origin/refactor/compiler-trim-one-
```

segment'.

To <https://github.com/avidullu/badminton-highlight-indexer.git>

* [new branch] refactor/compiler-trim-one-segment -> refactor/compiler-trim-one-segment

27. user (2026-06-18T08:12:51.323Z)

<https://github.com/avidullu/badminton-highlight-indexer/pull/224>

28. assistant (2026-06-18T08:12:53.115Z)

PR opened successfully. Let me confirm the PR URL.

29. user (2026-06-18T08:13:01.476Z)

Structured output provided successfully

30. assistant (2026-06-18T08:13:06.680Z)

Done. Pure behavior-preserving refactor of `backend/pipeline/stitcher/compiler.py` complete and PR #224 opened for owner review (not merged).

Summary:

- Extracted the per-segment loop body of `VideoCompiler.compile\_highlights` verbatim into a new instance method `\_trim\_one\_segment(...)` -> `(clip\_path\_or\_None, skip\_record\_or\_None)`.
- `compile\_highlights` keeps the for-loop and appends to `temp\_clips` / `skipped\_segments` exactly as before, calling the helper per segment.
- All four gates green: 856 passed / 7 skipped, ruff clean, mypy clean, coverage 80.76% (floor 70).
- Only `backend/pipeline/stitcher/compiler.py` was modified; branched from `origin/master`, staged the explicit path, verified HEAD before commit.
- PR: <https://github.com/avidullu/badminton-highlight-indexer/pull/224>